Announcements

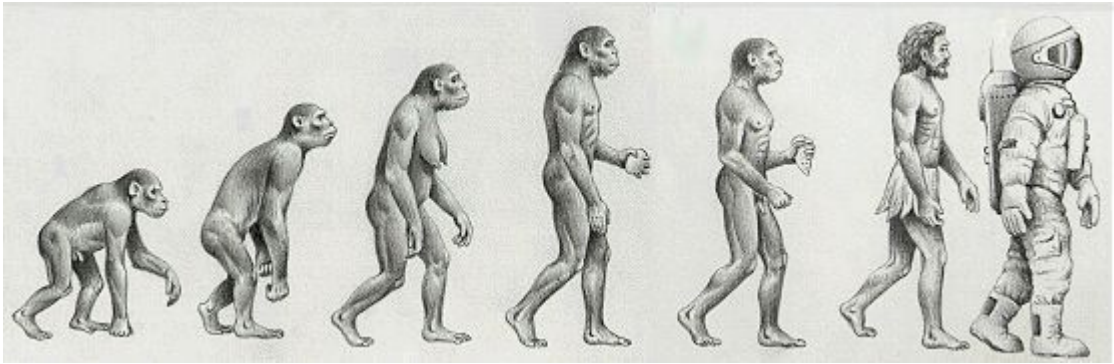
Project 1 was split into two pieces:

- Project 1A: Due this Friday.
  - Write tests for your LinkedListDeque.java.
- Project 1B: Due next Wednesday (one extra day).
  - Write LinkedListDeque.java.
  - You'll know what you need to know at the end of Friday's lecture.

It is too hot. (But Lake Anza is pretty nice)

A student bought an extra top dog. It is available, for free, to you: "A Hot Link"

CC BY NC SA



Lecture 6 (Lists 2)

SLLists, Nested Classes, Sentinel Nodes

CS61B, Fall 2026 @ UC Berkeley

Josh Hug and Manuel Sabin

CC BY NC SA

Getting Started: IntList → IntNode

Lecture 6, CS61B, Fall 2026

CC BY NC SA

Getting Started

- **`IntList` → `IntNode`**
- Creating the SLList Class
- **`addFirst`** and **`getFirst`**
- SLLists vs. IntLists

Syntax Improvements:

- Access Control: Public vs. Private
- Nested Classes

`addLast` and **`size`**

- Creating **`addLast`** and **`size`**
- **`size`** Efficiency, caching

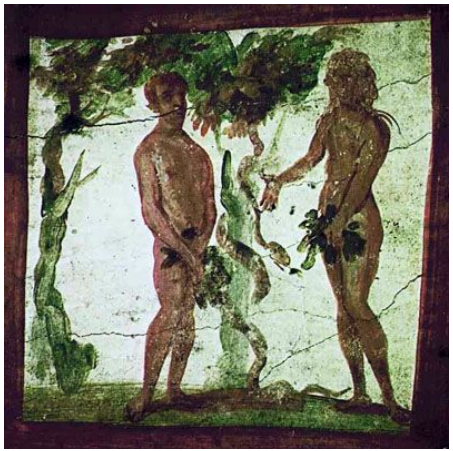
The Empty List

- **`addLast`** Bug
- Sentinel Nodes
- Invariants

Last Wednesday in 61B: Recursive Implementation of a List

```
public class IntList {
   public int first;
   public IntList rest;

   public IntList(int f, IntList r) {
      first = f;
      rest = r;
   }
   ...
}
```



While functional, “naked” linked lists like the one above are hard to use.

- Users of this class are probably going to need to know references very well, and be able to think recursively.
- Let’s make our users’ lives easier by **hiding the recursion**.

CC BY NC SA

Improvement #1: Rebranding and Culling

```
public class IntNode {
    public int item;
    public IntNode next;

    public IntNode(int i, IntNode n) {
        item = i;
        next = n;
    }

}
```

IntNode is now dumb, has no methods. We will reintroduce functionality in the coming slides.

Not much of an improvement obviously, but this next weird trick will be more impressive.

CC BY NC SA

Creating the SLList Class

Lecture 6, CS61B, Fall 2026

CC BY NC SA

Getting Started

- `IntList` → `IntNode`
- **Creating the SLList Class**
- `addFirst` and `getFirst`
- SLLists vs. IntLists

Syntax Improvements:

- Access Control: Public vs. Private
- Nested Classes

`addLast` and `size`

- Creating `addLast` and `size`
- `size` Efficiency, caching

The Empty List

- `addLast` Bug
- Sentinel Nodes
- Invariants

Coding Demo: Bureaucracy

SLList.java

```
/** An SLList is a list of integers, which hides the terrible truth
  * of the nakedness within. */
public class SLList {









}
```

CC BY NC SA

Coding Demo: Bureaucracy

SLList.java

```
/** An SLList is a list of integers, which hides the terrible truth
  * of the nakedness within. */
public class SLList {
   public IntNode first;









}
```

CC BY NC SA

Coding Demo: Bureaucracy

SLList.java

```
/** An SLList is a list of integers, which hides the terrible truth
  * of the nakedness within. */
public class SLList {
   public IntNode first;

   public SLList(int x) {

   }









}
```

CC BY NC SA

Coding Demo: Bureaucracy

SLList.java

```
/** An SLList is a list of integers, which hides the terrible truth
  * of the nakedness within. */
public class SLList {
   public IntNode first;

   public SLList(int x) {
      first = new IntNode(x, null);
   }









}
```

CC BY NC SA

Coding Demo: Bureaucracy

SLList.java

```
/** An SLList is a list of integers, which hides the terrible truth
  * of the nakedness within. */
public class SLList {
   public IntNode first;

   public SLList(int x) {
      first = new IntNode(x, null);
   }

   public static void main() {



   }




}
```

CC BY NC SA

Coding Demo: Bureaucracy

SLList.java

```
/** An SLList is a list of integers, which hides the terrible truth
  * of the nakedness within. */
public class SLList {
   public IntNode first;

   public SLList(int x) {
      first = new IntNode(x, null);
   }

   public static void main() {
      /** Creates a list of one integer, namely 10 */
      SLList L = new SLList(10);
   }



}
```

CC BY NC SA

Improvement #2: Bureaucracy

```
public class IntNode {
   public int item;
   public IntNode next;

   public IntNode(int i, IntNode n) {
      item = i;
      next = n;
   }

}
```

IntNode is now dumb, has no methods.

```
IntList X = new IntList(10, null);
SLList Y = new SLList(10);
```

SLList is easier to instantiate (no need to specify null), but we will see more advantages to come.

```
public class SLList {
   public IntNode first;

   public SLList(int x) {
      first = new IntNode(x, null);
   }
   ...
}
```

Next: Let's add addFirst and getFirst methods to SLList.

CC BY NC SA

addFirst and getFirst

Lecture 6, CS61B, Fall 2026

CC BY NC SA

Getting Started

- `IntList` → `IntNode`
- Creating the SLList Class
- **`addFirst` and `getFirst`**
- SLLists vs. IntLists

Syntax Improvements:

- Access Control: Public vs. Private
- Nested Classes

`addLast` and `size`

- Creating `addLast` and `size`
- `size` Efficiency, caching

The Empty List

- `addLast` Bug
- Sentinel Nodes
- Invariants

Coding Demo: addFirst and getFirst

SLList.java

```
public class SLList {
   public IntNode first;




}
```

CC BY NC SA

Coding Demo: addFirst and getFirst

SLList.java

```
public class SLList {
   public IntNode first;

   /** Adds x to the front of the list. */
   public void addFirst(int x) {

   }









}
```

CC BY NC SA

Coding Demo: addFirst and getFirst

SLList.java

```
public class SLList {
   public IntNode first;

   /** Adds x to the front of the list. */
   public void addFirst(int x) {
      first = new IntNode(x, first);
   }









}
```

This is how we added to the front of an IntList in the lecture 4.

CC BY NC SA

Coding Demo: addFirst and getFirst

SLList.java

```
public class SLList {
   public IntNode first;

   /** Adds x to the front of the list. */
   public void addFirst(int x) {
      first = new IntNode(x, first);
   }

   /** Returns the first item in the list. */
   public int getFirst() {

   }




}
```

CC BY NC SA

Coding Demo: addFirst and getFirst

SLList.java

```
public class SLList {
   public IntNode first;

   /** Adds x to the front of the list. */
   public void addFirst(int x) {
      first = new IntNode(x, first);
   }

   /** Returns the first item in the list. */
   public int getFirst() {
      return first.item;
   }




}
```

CC BY NC SA

Coding Demo: addFirst and getFirst

SLList.java

```
public class SLList {
   public IntNode first;

   public static void main() {
      SLList L = new SLList(15);
      L.addFirst(10);
      L.addFirst(5);
      IO.println(L.getFirst()); // should print 5
   }









}
```

CC BY NC SA

The Basic SLList and Helper IntNode Class

```
public class SLList {
   public IntNode first;

   public SLList(int x) {
      first = new IntNode(x, null);
   }

   public void addFirst(int x) {
      first = new IntNode(x, first);
   }

   public int getFirst() {
      return first.item;
   }
}
```

```
public class IntNode {
   public int item;
   public IntNode next;

   public IntNode(int i, IntNode n) {
       item = i;
       next = n;
   }

}
```

Example usage:

```
SLList L = new SLList(15);
L.addFirst(10);
L.addFirst(5);
int x = L.getFirst();
```

CC BY NC SA

SLLists vs. IntLists

Lecture 6, CS61B, Fall 2026

CC BY NC SA

Getting Started

- `IntList` → `IntNode`
- Creating the SLList Class
- `addFirst` and `getFirst`
- **SLLists vs. IntLists**

Syntax Improvements:

- Access Control: Public vs. Private
- Nested Classes

`addLast` and `size`

- Creating `addLast` and `size`
- `size` Efficiency, caching

The Empty List

- `addLast` Bug
- Sentinel Nodes
- Invariants

SLLists vs. IntLists

```
SLList L = new SLList(15);
L.addFirst(10);
L.addFirst(5);
int x = L.getFirst();
```

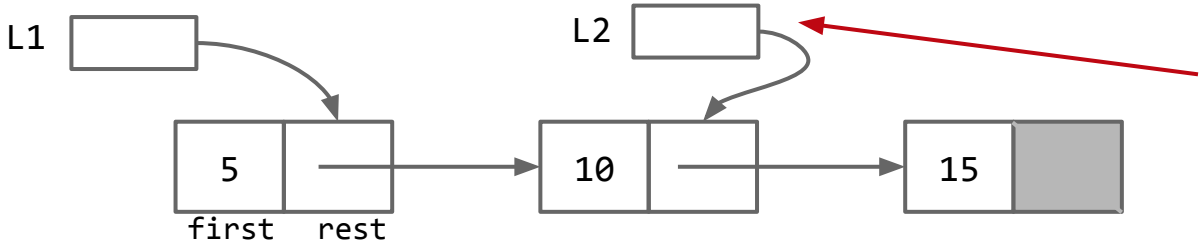
```
IntList L = new IntList(15, null);
L = new IntList(10, L);
L = new IntList(5, L);
int x = L.first;
```

While functional, “naked” linked lists like the `IntList` class are hard to use.

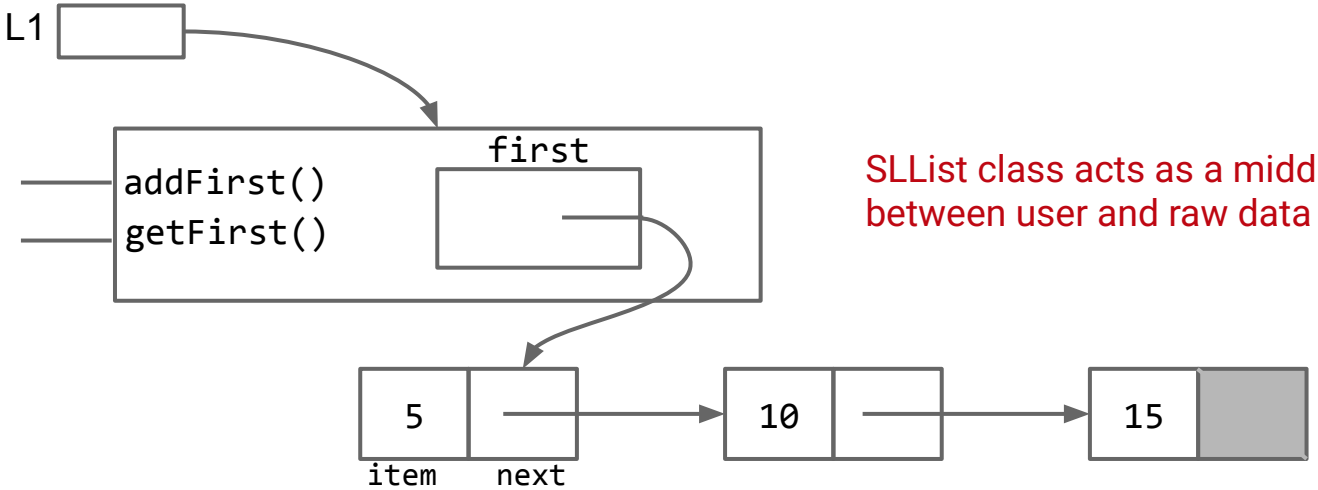
- Users of `IntList` need to know Java references well, and be able to think recursively.
- `SLList` is much simpler to use. Simply use the provided methods.
- Why not just add an `addFirst` method to the `IntList` class? Turns out there is no efficient way to do this. Try it out and you’ll see it’s hard (and inefficient).

CC BY NC SA

Naked Linked Lists (IntList) vs. SLLists



Naked recursion: Natural for IntList user to have variables that point to the middle of the IntList.



SLList class acts as a middle man between user and raw data structure.

CC BY NC SA

Access Control: Public vs. Private

Lecture 6, CS61B, Fall 2026

Getting Started

- `IntList` → `IntNode`
- Creating the SLList Class
- `addFirst` and `getFirst`
- SLLists vs. IntLists

Syntax Improvements:

- **Access Control: Public vs. Private**
- Nested Classes

`addLast` and `size`

- Creating `addLast` and `size`
- `size` Efficiency, caching

The Empty List

- `addLast` Bug
- Sentinel Nodes
- Invariants

CC BY NC SA

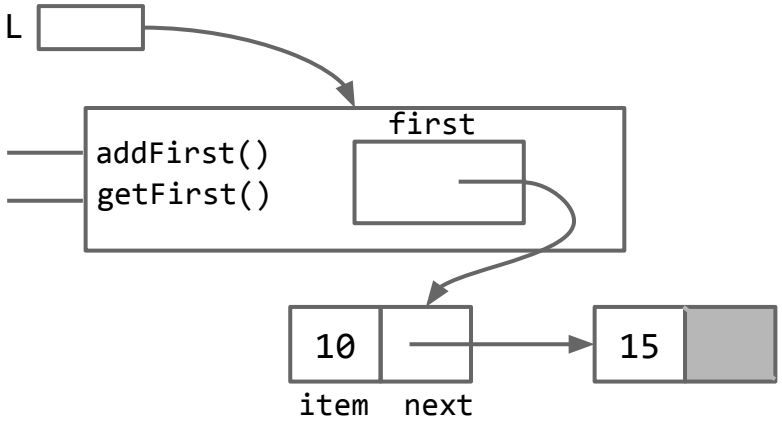
The SLList So Far

```
public class SLList {
   public IntNode first;

   public SLList(int x) {
      first = new IntNode(x, null);
   }

   public void addFirst(int x) {
      first = new IntNode(x, first);
   }

   ...
}
```




```
SLList L = new SLList(15);
L.addFirst(10);
```

CC BY NC SA

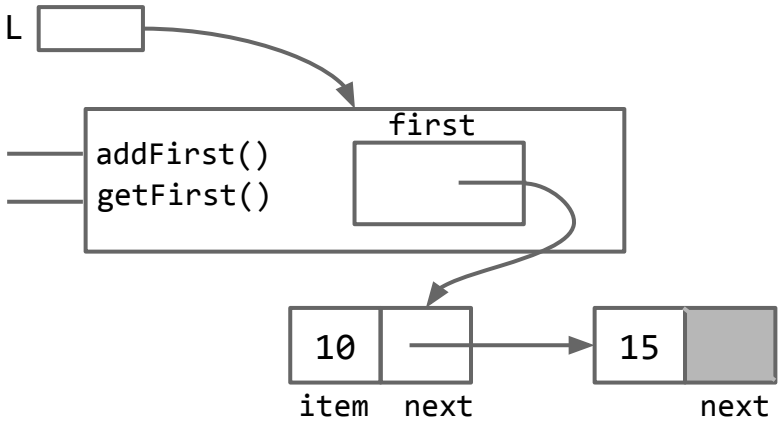
A Potential SLList Danger

```
public class SLList {
   public IntNode first;

   public SLList(int x) {
      first = new IntNode(x, null);
   }

   public void addFirst(int x) {
      first = new IntNode(x, first);
   }

   ...
}
```



Users of our class might be tempted to try to manipulate our secret IntNode directly in uncouth ways!

```
SLList L = new SLList(15);
L.addFirst(10);
L.first.next.next = L.first.next;
```

CC BY NC SA

A Potential SLList Danger

```
public class SLList {
    public IntNode first;

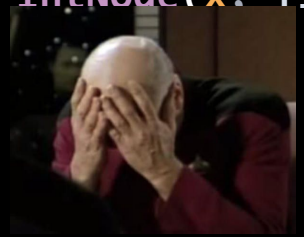
    publ                x) {
        f           tNode
    }

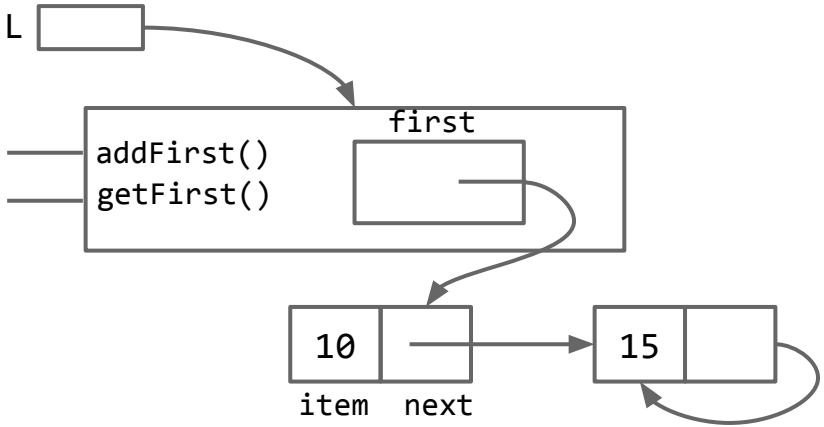
    public void addFirst(int x) {
        first = new IntNode(x, first);
    }
}
```











Users of our class might be tempted to try to manipulate our secret IntNode directly in uncouth ways!

```
SLList L = new SLList(15);
L.addFirst(10);
L.first.next.next = L.first.next;
```

CC BY NC SA

Access Control

```
public class SLList {
   public IntNode first;

   public SLList(int x) {
      first = new IntNode(x, null);
   }

   public void addFirst(int x) {
      first = new IntNode(x, first);
   }

   ...
}
```

We can prevent programmers from making such mistakes with the **`private`** keyword.

CC BY NC SA

Improvement #3: Access Control

```
public class SLList {
   private IntNode first;

   public SLList(int x) {
      first = new IntNode(x, null);
   }

   public void addFirst(int x) {
      first = new IntNode(x, first);
   }

   ...
}
```

Use the **private** keyword to prevent code in other classes from using members (or constructors) of a class.

```
SLList L = new SLList(15);
L.addFirst(10);
L.first.next.next = L.first.next;
```

```
jug ~/Dropbox/61b/lec/lists2
$ javac SLListUser.java
SLListUser.java:8: error: first has private access in SLList
        L.first.next.next = L.first.next;
```

CC BY NC SA

Why Restrict Access?

Hide implementation details from users of your class.

- Less for user of class to understand.
- Safe for you to change private methods (implementation).

Car analogy:

- **Public**: Pedals, Steering Wheel **Private**: Fuel line, Rotary valve

- Despite the term ‘access control’:
  - Nothing to do with protection against hackers, spies, LLMs, and other evil entities.

CC BY NC SA

Nested Classes

Lecture 6, CS61B, Fall 2026

Getting Started

- `IntList` → `IntNode`
- Creating the SLList Class
- `addFirst` and `getFirst`
- SLLists vs. IntLists

Syntax Improvements:

- Access Control: Public vs. Private
- **Nested Classes**

`addLast` and `size`

- Creating `addLast` and `size`
- `size` Efficiency, caching

The Empty List

- `addLast` Bug
- Sentinel Nodes
- Invariants

CC BY NC SA

Improvement #4: Nested Classes

Can combine two classes into one file pretty simply.

```
public class SLList {
    public class IntNode {
        public int item;
        public IntNode next;
        public IntNode(int i, IntNode n) {
            item = i;
            next = n;
        }
    }

    private IntNode first;
    public SLList(int x) {
        first = new IntNode(x, null);
    } ...
```

Nested class definition.

Could have made `IntNode` a private nested class if we wanted.

Instance variables, constructors, and methods of `SLList` typically go below nested class definition.

CC BY NC SA

Why Nested Classes?

Nested Classes are useful when a class doesn't stand on its own and is obviously subordinate to another class.

- Make the nested class private if other classes should never use the nested class.

In my opinion, probably makes sense to make `IntNode` a nested `private` class.

- Hard to imagine other classes having a need to manipulate `IntNodes`.
- If there was some hypothetical strange function like:
  ```
  public IntNode getFrontNode()
  ```
  Then we would need the `IntNode` class to be public.

CC BY NC SA

Static Nested Classes (Extra)

If the nested class never uses any instance variables or methods of the outer class, declare it static.

- Static classes cannot access outer class's instance variables or methods.
- Results in a minor savings of memory. See book for more details / exercise.

```
public class SLList {
   private static class IntNode {
      public int item;
      public IntNode next;
      public IntNode(int i, IntNode n) {
         item = i;
         next = n;
      }
   ...
}
```

We can declare IntNode static, since it never uses any of SLList's instance variables or methods.

Analogy: Static methods had no way to access "my" instance variables. Static classes cannot access "my" outer class's instance variables.

Unimportant note: For private nested classes, access modifiers are irrelevant.

CC BY NC SA

Creating addLast and size

Lecture 6, CS61B, Fall 2026

Getting Started

- `IntList` → `IntNode`
- Creating the SLList Class
- `addFirst` and `getFirst`
- SLLists vs. IntLists

Syntax Improvements:

- Access Control: Public vs. Private
- Nested Classes

`addLast` and `size`

- **Creating `addLast` and `size`**
- `size` Efficiency, caching

The Empty List

- `addLast` Bug
- Sentinel Nodes
- Invariants

CC BY NC SA

Adding More SLList Functionality

To motivate our remaining improvements, and to give more functionality to our `SLList` class, let's add:

- `.addLast(int x)`
- `.size()`



Try it in your browser:

- hugcode.com/61b/lecs/lec6

Recommendation: Try writing them yourself before watching how I do it.

Methods	Non-Obvious Improvements	
`addFirst(int x)`	#1	Rebranding: `IntList` → `IntNode`
`getFirst`	#2	Bureaucracy: `SLList`
	#3	Access Control: `public` → `private`
	#4	Nested Class: Bringing `IntNode` into `SLList`

CC BY NC SA

Coding Demo: addLast

SLList.java

```
public class SLList {
   private IntNode first;

   /** Adds an item to the end of the list. */
   public void addLast(int x) {









   }


}
```

CC BY NC SA

Coding Demo: addLast

SLList.java

```
public class SLList {
   private IntNode first;

   /** Adds an item to the end of the list. */
   public void addLast(int x) {
      IntNode p = first;









   }


}
```

CC BY NC SA

Coding Demo: addLast

SLList.java

```
public class SLList {
   private IntNode first;

   /** Adds an item to the end of the list. */
   public void addLast(int x) {
      IntNode p = first;


      while (p.next != null) {

      }


   }


}
```

CC BY NC SA

Coding Demo: addLast

SLList.java

```
public class SLList {
   private IntNode first;

   /** Adds an item to the end of the list. */
   public void addLast(int x) {
      IntNode p = first;


      while (p.next != null) {
         p = p.next;
      }


   }


}
```

CC BY NC SA

Coding Demo: addLast

SLList.java

```
public class SLList {
   private IntNode first;

   /** Adds an item to the end of the list. */
   public void addLast(int x) {
      IntNode p = first;

      /* Move p until it reaches the end of the list. */
      while (p.next != null) {
         p = p.next;
      }


   }


}
```

CC BY NC SA

Coding Demo: addLast

SLList.java

```
public class SLList {
   private IntNode first;

   /** Adds an item to the end of the list. */
   public void addLast(int x) {
      IntNode p = first;

      /* Move p until it reaches the end of the list. */
      while (p.next != null) {
         p = p.next;
      }

      p.next = new IntNode(x, null);
   }


}
```

CC BY NC SA

Coding Demo: addLast

SLList.java

```
public class SLList {
   private IntNode first;

   public static void main() {
      SLList L = new SLList(15);
      L.addFirst(10);
      L.addFirst(5);
      L.addLast(20);
      IO.println(L.getFirst()); // should print 5
   }
}
```

Running this code doesn't actually prove that our addLast works! It's just a single test.

Java visualizer

CC BY NC SA

Coding Demo: size

SLList.java

```
public class SLList {
   private IntNode first;

   public int size() {

   }



}
```

CC BY NC SA

Coding Demo: size

SLList.java

```
public class SLList {
   private IntNode first;

   public int size() {

   }



}
```

Writing a recursive size method is tricky, because SLList itself is not recursive.

The size method doesn't take in any arguments, so calling size recursively is strange. What is the base case? How do you call size recursively to get closer to the base case?

CC BY NC SA

Coding Demo: size

SLList.java

```
public class SLList {
   private IntNode first;


   private static int size(IntNode p) {









   }

   public int size() {

   }
}
```

Solution: Write a private static helper method that takes in an extra argument to help with recursion.

(This can be static because we don't need to reference first.)

CC BY NC SA

Coding Demo: size

SLList.java

```
public class SLList {
   private IntNode first;

   /** Returns the size of the list that starts at IntNode p. */
   private static int size(IntNode p) {









   }

   public int size() {

   }
}
```

CC BY NC SA

Coding Demo: size

SLList.java

```
public class SLList {
   private IntNode first;

   /** Returns the size of the list that starts at IntNode p. */
   private static int size(IntNode p) {
      if (p.next == null) {

      }


   }

   public int size() {

   }
}
```

CC BY NC SA

Coding Demo: size

SLList.java

```
public class SLList {
   private IntNode first;

   /** Returns the size of the list that starts at IntNode p. */
   private static int size(IntNode p) {
      if (p.next == null) {
         return 1;
      }


   }

   public int size() {

   }
}
```

CC BY NC SA

Coding Demo: size

SLList.java

```
public class SLList {
   private IntNode first;

   /** Returns the size of the list that starts at IntNode p. */
   private static int size(IntNode p) {
      if (p.next == null) {
         return 1;
      }

      return 1 + size(p.next);
   }

   public int size() {

   }
}
```

CC BY NC SA

Coding Demo: size

SLList.java

```
public class SLList {
   private IntNode first;

   /** Returns the size of the list that starts at IntNode p. */
   private static int size(IntNode p) {
      if (p.next == null) {
         return 1;
      }

      return 1 + size(p.next);
   }

   public int size() {
      return size(first);
   }
}
```

CC BY NC SA

Private Recursive Helper Methods

To implement a recursive method in a class that is not itself recursive (e.g. SLList):

- Create a private recursive helper method.
- Have the public method call the private recursive helper method.

```
public class SLList {
  private IntNode first;

  private static int size(IntNode p) {
     if (p.next == null) {
        return 1;
     }

     return 1 + size(p.next);
  }

  public int size() {
     return size(first);
  }
}
```

CC BY NC SA

size Efficiency, caching

Lecture 6, CS61B, Fall 2026

Getting Started

- `IntList` → `IntNode`
- Creating the SLList Class
- `addFirst` and `getFirst`
- SLLists vs. IntLists

Syntax Improvements:

- Access Control: Public vs. Private
- Nested Classes

`addLast` and `size`

- Creating `addLast` and `size`
- **`size` Efficiency, caching**

The Empty List

- `addLast` Bug
- Sentinel Nodes
- Invariants

CC BY NC SA

Attendance Question: Efficiency of Size: hugcode.com/mango

How efficient is size?

- Suppose size takes 2 seconds on a list of size 1,000.
- How long will it take on a list of size 1,000,000?

a. 0.002 seconds.
b. 2 seconds.
c. 2,000 seconds.
d. 2,000,000 seconds.



```
public class SLList {
   private IntNode first;

   private int size(IntNode p) {
      if (p.next == null) {
         return 1;
      }

      return 1 + size(p.next);
   }

   public int size() {
      return size(first);
   }
}
```

CC BY NC SA

Improvement #5: Fast size()

Your goal:

- Modify SLList so that the execution time of size() is always fast (i.e. independent of the size of the list).

(video viewers only, time is too tight in class to think carefully about this)

```
public class SLList {
   private IntNode first;

   public SLList(int x) {
      first = new IntNode(x, null);
   }

   public void addFirst(int x) {
      first = new IntNode(x, first);
   }

   private int size(IntNode p) {
      if (p.next == null)
         return 1;
      return 1 + size(p.next);
   }

   public int size() {
      return size(first);
   }
}
```

CC BY NC SA

Coding Demo: Fast size

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;
}
```

Instead of re-calculating size on demand every time size is called, we'll keep track of the current size in a private variable.

Then, we'll update the size variable every time the list is changed.

This variable is redundant (we could calculate the size from the list), but will save us time.

CC BY NC SA

Coding Demo: Fast size

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   public SLList(int x) {
      first = new IntNode(x, null);

   }









}
```

CC BY NC SA

Coding Demo: Fast size

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   public SLList(int x) {
      first = new IntNode(x, null);
      size = 1;
   }









}
```

New line added to existing function.

CC BY NC SA

Coding Demo: Fast size

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   /** Adds x to the front of the list. */
   public void addFirst(int x) {
      first = new IntNode(x, first);

   }









}
```

CC BY NC SA

Coding Demo: Fast size

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   /** Adds x to the front of the list. */
   public void addFirst(int x) {
      first = new IntNode(x, first);
      size += 1;
   }









}
```

New line added to existing function.

CC BY NC SA

Coding Demo: Fast size

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   /** Returns the first item in the list. */
   public int getFirst() {
      return first.item;
   }









}
```

No modification needed. getFirst doesn't change the size of the list.

CC BY NC SA

Coding Demo: Fast size

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   /** Adds x to the end of the list. */
   public void addLast(int x) {

      IntNode p = first;

      /** Move p until it reaches the end of the list. */
      while (p.next != null) {
         p = p.next;
      }

      p.next = new IntNode(x, null);
   }
}
```

CC BY NC SA

Coding Demo: Fast size

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   /** Adds x to the end of the list. */
   public void addLast(int x) {
      size += 1;
      IntNode p = first;

      /** Move p until it reaches the end of the list. */
      while (p.next != null) {
         p = p.next;
      }

      p.next = new IntNode(x, null);
   }
}
```

New line added to existing function.

CC BY NC SA

Coding Demo: Fast size

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   public int size() {
      return size;
   }









}
```

CC BY NC SA

Improvement #5: Fast size()

Solution: Maintain a special size variable that **caches** the size of the list.

- Caching: putting aside data to speed up retrieval.

TANSTAAFL: There ain't no such thing as a free lunch.

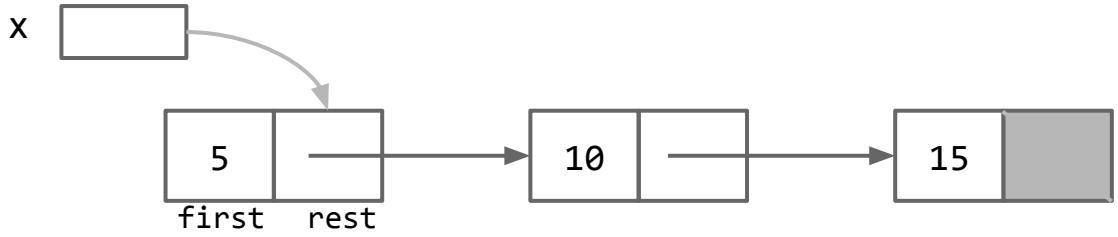
- But spreading the work over each add call is a net win in almost any circumstance.

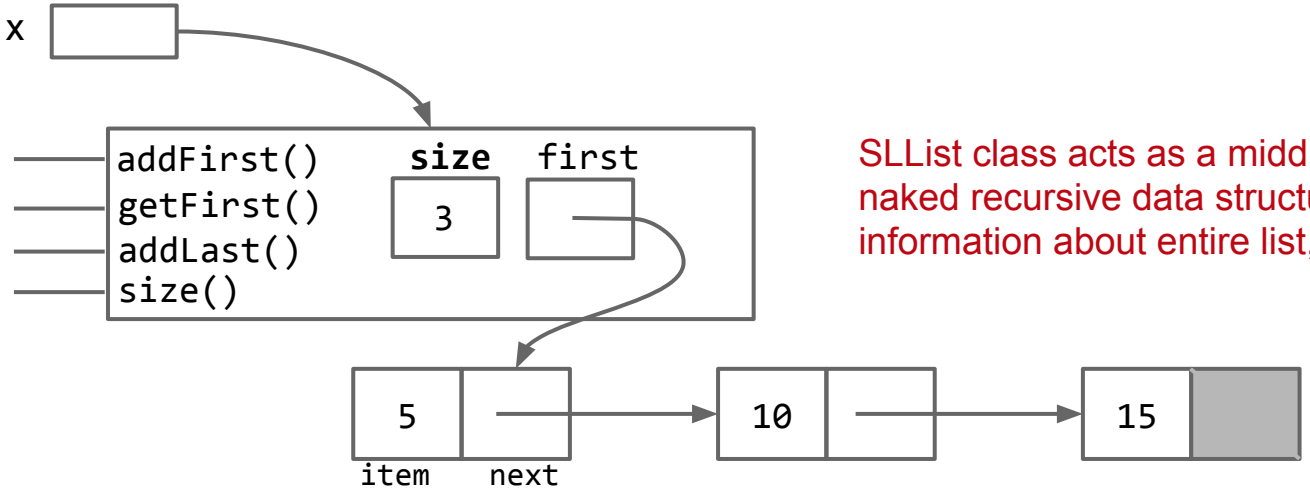


http://www.ensler.us/ensler.us/images/nolnchsmalla.jpg

CC BY NC SA

Naked Linked Lists (IntList) vs. SLLists





SLList class acts as a middle man between user and the naked recursive data structure. Allows us to store meta information about entire list, e.g. **`size`**.

CC BY NC SA

The Empty List and the addLast Bug

Lecture 6, CS61B, Fall 2026

Getting Started

- `IntList` → `IntNode`
- Creating the SLList Class
- `addFirst` and `getFirst`
- SLLists vs. IntLists

Syntax Improvements:

- Access Control: Public vs. Private
- Nested Classes

`addLast` and `size`

- Creating `addLast` and `size`
- `size` Efficiency, caching

The Empty List

- **`addLast` Bug**
- Sentinel Nodes
- Invariants

CC BY NC SA

Coding Demo: The Empty List

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   public SLList(int x) {
      first = new IntNode(x, null);
      size = 1;
   }

   /** Creates an empty SLList. */
   public SLList() {



   }
}
```

Adding a second constructor for empty lists (in addition to the existing constructor for making lists with 1 element).

CC BY NC SA

Coding Demo: The Empty List

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   public SLList(int x) {
      first = new IntNode(x, null);
      size = 1;
   }

   /** Creates an empty SLList. */
   public SLList() {

      size = 0;
   }
}
```

Adding a second constructor for empty lists (in addition to the existing constructor for making lists with 1 element).

CC BY NC SA

Coding Demo: The Empty List

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   public SLList(int x) {
      first = new IntNode(x, null);
      size = 1;
   }

   /** Creates an empty SLList. */
   public SLList() {
      first = null;
      size = 0;
   }
}
```

Adding a second constructor for empty lists (in addition to the existing constructor for making lists with 1 element).

CC BY NC SA

Coding Demo: The Empty List

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   public static void main() {
      SLList L = new SLList();
      L.addFirst(10);
      L.addFirst(5);
      L.addLast(20);
      IO.println(L.getFirst()); // should print 5
   }



}
```

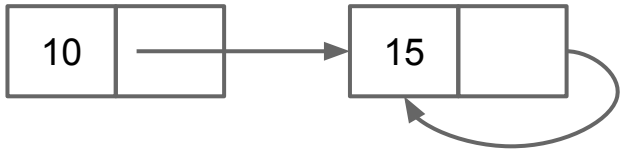
Java visualizer

CC BY NC SA

Improvement #6a: Representing the Empty List

Benefits of `SLList` vs. `IntList` so far:

- Faster `size()` method than would have been convenient for `IntList`.
- User of an `SLList` never sees the `IntList` class.
  - Simpler to use.
  - More efficient `addFirst` method (see exercises).
  - Avoids errors (or malfeasance):



Another benefit we can gain:

- Easy to represent the empty list. Represent the empty list by setting `first` to null. Let's try!
- We'll see there is a **very** subtle bug in the code. It crashes when you call addLast on an empty list.

CC BY NC SA

Coding Demo: The addLast Bug

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   public static void main() {
      SLList L = new SLList();
      L.addLast(20); // program crashes!
   }




}
```

Java visualizer

CC BY NC SA

Coding Demo: The addLast Bug

SLList.java

```
public class SLList {
   private IntNode first;
   private int size;

   /** Adds x to the end of the list. */
   public void addLast(int x) {
      size += 1;
      IntNode p = first;

      /** Move p until it reaches the end of the list. */
      while (p.next != null) {
         p = p.next;
      }

      p.next = new IntNode(x, null);
   }
}
```

The NullPointerException traceback identifies this line as where the program crashed.

CC BY NC SA

How Would You Fix addLast?

Your goal:

- Fix addLast so that we do not get a null pointer exception when we try to add to the back of an empty `SLList`:

```
SLList s1 = new SLList();
s1.addLast(5);
```

Try it in your browser:
hugcode.com/61b/lecs/lec6b



```
public class SLList {
   private IntNode first;
   private int size;

   public SLList() {
      first = null;
      size = 0;
   }

   public void addLast(int x) {
      size += 1;
      IntNode p = first;
      while (p.next != null) {
         p = p.next;
      }

      p.next = new IntNode(x, null);
   }
}
```

CC BY NC SA

One Solution

One possible solution:

- Add a special case for the empty list.

But there are other ways...

```
public void addLast(int x) {
   size += 1;

   if (first == null) {
      first = new IntNode(x, null);
      return;
   }

   IntNode p = first;
   while (p.next != null) {
      p = p.next;
   }

   p.next = new IntNode(x, null);
}
```

cc BY NC SA

Sentinel Nodes

Lecture 6, CS61B, Fall 2026

Getting Started

- `IntList` → `IntNode`
- Creating the SLList Class
- `addFirst` and `getFirst`
- SLLists vs. IntLists

Syntax Improvements:

- Access Control: Public vs. Private
- Nested Classes

`addLast` and `size`

- Creating `addLast` and `size`
- `size` Efficiency, caching

The Empty List

- `addLast` Bug
- **Sentinel Nodes**
- Invariants

CC BY NC SA

Tip For Being a Good Programmer: Keep Code Simple

As a human programmer, you only have so much working memory.

- You want to restrict the amount of complexity in your life!
- Simple code is (usually) good code.
  - Special cases are not ‘simple’.



```
public void addLast(int x) {
   size += 1;

   if (first == null) {
      first = new IntNode(x, null);
      return;
   }

   IntNode p = first;
   while (p.next != null) {
      p = p.next;
   }

   p.next = new IntNode(x, null);
}
```

cc BY NC SA

addLast's Fundamental Problem

The fundamental problem:

- The empty list has a null **`first`**. Can't access `first.next`!

Our fix is a bit ugly:

- Requires a special case.
- More complex data structures will have many more special cases (gross!!)

How can we avoid special cases?

- Make all `SLLists` (even empty) the "same".

```
public void addLast(int x) {
   size += 1;

   if (first == null) {
      first = new IntNode(x, null);
      return;
   }

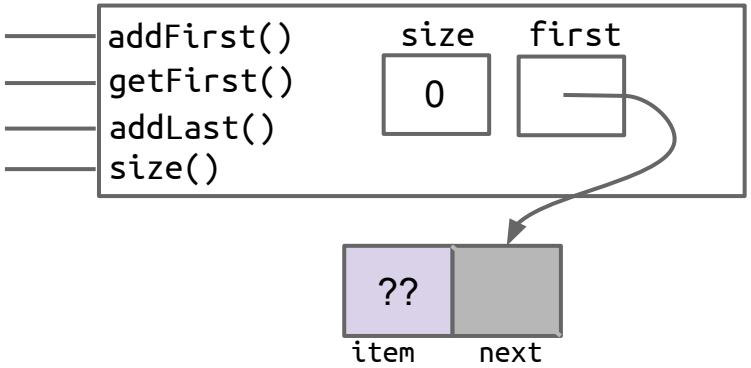
   IntNode p = first;
   while (p.next != null) {
      p = p.next;
   }

   p.next = new IntNode(x, null);
}
```

CC BY NC SA

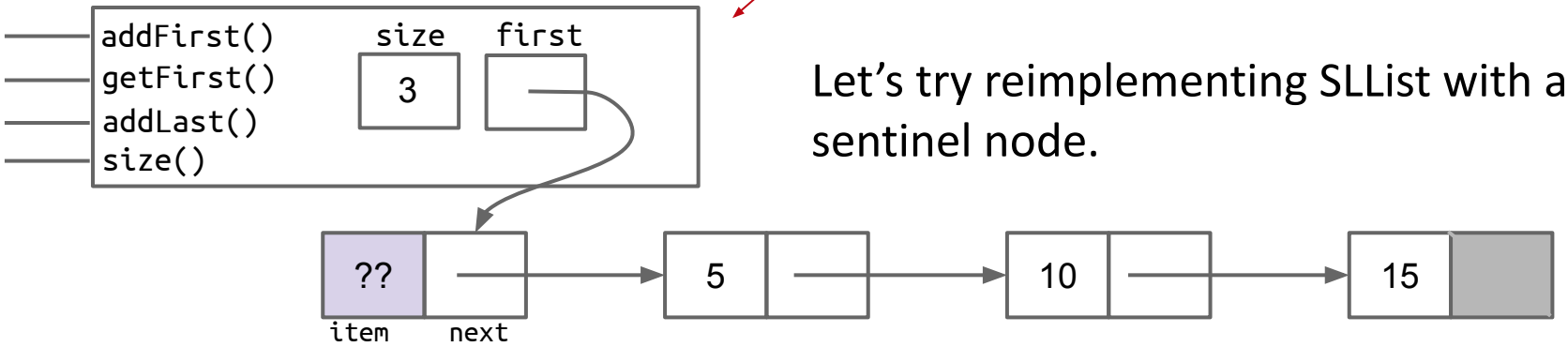
Improvement #6b: Representing the Empty List Using a Sentinel

Create a special node that is always there! Let's call it a "sentinel node".



The empty list is just the sentinel node.

A list with 3 numbers has a sentinel node and 3 nodes that contain real data.



Let's try reimplementing SLList with a sentinel node.

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {

   private IntNode first;
   private int size;

   public SLList(int x) {

      first = new IntNode(x, null);
      size = 1;
   }

   public SLList() {
      first = null;
      size = 0;
   }

}
```

In this demo, we'll be modifying our existing code to account for the sentinel node.

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {
   /** The first item (if it exists) is at sentinel.next. */
   private IntNode sentinel;
   private int size;

   public SLList(int x) {

      first = new IntNode(x, null);
      size = 1;
   }

   public SLList() {
      first = null;
      size = 0;
   }

}
```

Renaming first to sentinel.

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {
   /** The first item (if it exists) is at sentinel.next. */
   private IntNode sentinel;
   private int size;

   public SLList(int x) {

      first = new IntNode(x, null);
      size = 1;
   }

   public SLList() {
      sentinel = new IntNode(63, null);
      size = 0;
   }

}
```

The empty list is no longer null. It's the sentinel node (and no other nodes).

63 is a placeholder. We don't care about the sentinel node's value.

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {
   /** The first item (if it exists) is at sentinel.next. */
   private IntNode sentinel;
   private int size;

   public SLList(int x) {
      sentinel = new IntNode(63, null);
      sentinel.next = new IntNode(x, null);
      size = 1;
   }

   public SLList() {
      sentinel = new IntNode(63, null);
      size = 0;
   }

}
```

The single-element list is no longer just one node. It's two nodes: the sentinel node, followed by the node with the single value.

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {
   /** The first item (if it exists) is at sentinel.next. */
   private IntNode sentinel;
   private int size;

   /** Adds x to the front of the list. */
   public void addFirst(int x) {
      first = new IntNode(x, first);
      size += 1;
   }









}
```

Need to modify addFirst to be consistent with the sentinel structure.

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {
   /** The first item (if it exists) is at sentinel.next. */
   private IntNode sentinel;
   private int size;

   /** Adds x to the front of the list. */
   public void addFirst(int x) {
      sentinel.next = new IntNode(x, sentinel.next);
      size += 1;
   }









}
```

We need to reassign sentinel.next here, because "the first item is at sentinel.next."

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {
   /** The first item (if it exists) is at sentinel.next. */
   private IntNode sentinel;
   private int size;

   /** Returns the first item in the list. */
   public int getFirst() {
      return first.item;
   }









}
```

Need to modify getFirst to be consistent with the sentinel structure.

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {
   /** The first item (if it exists) is at sentinel.next. */
   private IntNode sentinel;
   private int size;

   /** Returns the first item in the list. */
   public int getFirst() {
      return sentinel.next.item;
   }

}
```

Recall: "the first item is at sentinel.next."

If we returned sentinel.item, we would get the placeholder 63, not the true first item of the list.

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {
   /** The first item (if it exists) is at sentinel.next. */
   private IntNode sentinel;
   private int size;

   /** Adds x to the end of the list. */
   public void addLast(int x) {
      size += 1;
      IntNode p = first;

      /** Move p until it reaches the end of the list. */
      while (p.next != null) {
         p = p.next;
      }
      p.next = new IntNode(x, null);
   }
}
```

Need to modify addLast to be consistent with the sentinel structure.

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {
   /** The first item (if it exists) is at sentinel.next. */
   private IntNode sentinel;
   private int size;

   /** Adds x to the end of the list. */
   public void addLast(int x) {
      size += 1;
      IntNode p = sentinel;

      /** Move p until it reaches the end of the list. */
      while (p.next != null) {
         p = p.next;
      }
      p.next = new IntNode(x, null);
   }
}
```

Start p at the sentinel, and move p until it reaches the end of the list.

Sentinel always exists, so the addLast bug doesn't apply anymore.

CC BY NC SA

Coding Demo: Sentinel Nodes

SLList.java

```
public class SLList {
   /** The first item (if it exists) is at sentinel.next. */
   private IntNode sentinel;
   private int size;

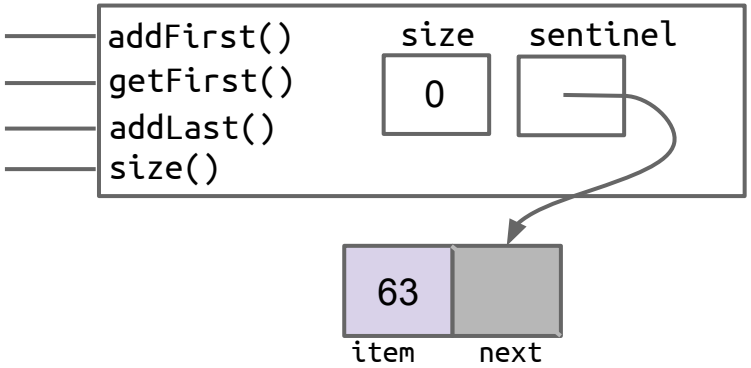
   public static void main() {
      SLList L = new SLList();
      L.addLast(20);
      IO.println(L.size()); //should print out 1
   }
}
```

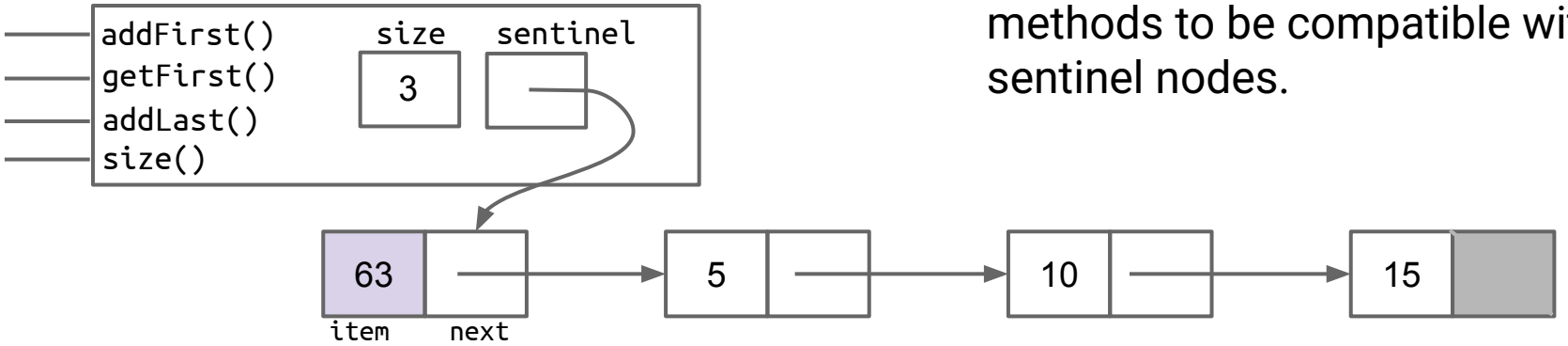
Java visualizer

CC BY NC SA

Sentinel Node

The sentinel node is always there for you.





Notes:

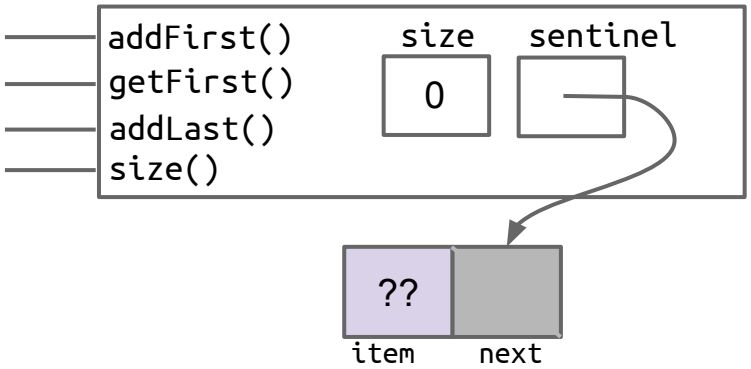
- I've renamed `first` to be `sentinel`.
- `sentinel` is never null, always points to sentinel node.
- Sentinel node's `item` needs to be some integer, but doesn't matter what value we pick.
- Had to fix constructors and methods to be compatible with sentinel nodes.

CC BY NC SA

addLast (with Sentinel Node)

Bottom line: Having a sentinel simplifies our `addLast` method.

- No need for a special case to check if `sentinel` is null (since it is never null).




```
public void addLast(int x) {
   size += 1;

   if (sentinel == null) {
      sentinel = new IntNode(x, null);
      return;
   }

   IntNode p = sentinel;
   while (p.next != null) {
      p = p.next;
   }

   p.next = new IntNode(x, null);
}
```

CC BY NC SA

Invariants

Lecture 6, CS61B, Fall 2026

Getting Started

- `IntList` → `IntNode`
- Creating the SLList Class
- `addFirst` and `getFirst`
- SLLists vs. IntLists

Syntax Improvements:

- Access Control: Public vs. Private
- Nested Classes

`addLast` and `size`

- Creating `addLast` and `size`
- `size` Efficiency, caching

The Empty List

- `addLast` Bug
- Sentinel Nodes
- **Invariants**

CC BY NC SA

Invariants

An invariant is a condition that is guaranteed to be true during code execution (assuming there are no bugs in your code).

An `SLList` with a sentinel node has at least the following invariants:

- The `sentinel` reference always points to the sentinel node.
- The first node (if it exists), is always at `sentinel.next`.
- The `size` variable is always the total number of items that have been added.

Invariants make it easier to reason about code:

- Can assume they are true to simplify code (e.g. `addLast` doesn't need to worry about nulls).
- Must ensure that methods preserve invariants.

CC BY NC SA

Summary

Methods	Non-Obvious Improvements	
`addFirst(int x)`	#1	Rebranding: `IntList` → `IntNode`
`getFirst`	#2	Bureaucracy: `SLList`
`size`	#3	Access Control: `public` → `private`
`addLast(int x)`	#4	Nested Class: Bringing `IntNode` into `SLList`
	#5	Caching: Saving `size` as an `int`.
	#6	Generalizing: Adding a `sentinel` node to allow representation of the empty list.

cc BY NC SA

Reflections on ~~IntList~~ SLList

The Good:

- Doesn't have a fixed length!
- Implementation was simple-ish.
- Usage is very simple.

The Bad:

- Slow to get to the end of the list!
- Only stores ints.
- Slow to access items in the middle of the list.

```
public class SLList {
   /** The first item (if it exists)
    *is at sentinel.next. */
   private IntNode sentinel;
   private int size;

   /** Adds x to the front of the list. */
   public void addFirst(int x) {
      sentinel.next = new IntNode(x, sentinel.next);
      size += 1;
   }

   ...
}
```

CC BY NC SA

Next up: Lab 3

If today felt like a lot to digest, don’t worry.

- Lab this week gives you additional practice with recursive lists.
- Project 1B gives you a chance to digest (due next week).

CC BY NC SA